PYSPARK COMPLETE NOTES

PHASE 2

Beginner Transformations

Select, filter, rename, create and clean columns, sort, limit and remove duplicates.

YOU WILL

- ✓ Choose, rename and drop columns
- ✓ Filter rows with &, |, isNull() and isin()
- ✓ Add and replace columns with withColumn, lit and regexp_replace
- ✓ Cast types, sort on several columns and take the top n
- ✓ Remove duplicate rows or duplicates by a subset of columns

START HERE IF

Phase 1 (reading a CSV into the BigMart DataFrame).

0 1 **2** 3 4 5 6 7 A

PHASES 0–7 APPENDICES

Built from a Databricks PySpark tutorial notebook and its course outline.
Clickable contents, links and bookmarks work in any PDF viewer.

Phase 2 · Beginner Transformations

Select, filter, rename, create and clean columns, sort, limit and remove duplicates.

YOU WILL BE ABLE TO

- ✓ Choose, rename and drop columns
- ✓ Filter rows with &, |, isNull() and isin()
- ✓ Add and replace columns with withColumn, lit and regexp_replace
- ✓ Cast types, sort on several columns and take the top n
- ✓ Remove duplicate rows or duplicates by a subset of columns

START HERE IF

Phase 1 (reading a CSV into the BigMart DataFrame).

IN THIS PDF

Explanations, diagrams and code from the notebook · sections tagged **Beyond the notebook** · a decoder for every term · interview questions · a data-engineering lens · a self-test with answers · a glossary

How to read the boxes

IN PLAIN WORDS A jargon-free explanation of the idea.	**DECODE THE CODE** Meaning of every function, option and symbol in the code above (first time it appears).
TIP A habit that makes your code cleaner, faster or safer.	**WATCH OUT** A trap that gives a wrong or surprising result.
NOTEBOOK OBSERVATION Something specific noticed in the course notebook.	**DATA ENGINEERING LENS** How the skill is used in real pipelines (summary page at the end).
EXTRA Extra knowledge that is not in the notebook.	**BEYOND THE NOTEBOOK** A tag under a heading: new material added to the notebook's topics, ending with an Interview corner.

CONTENTS (click an entry to jump to it)

2.1 select() . . . 3
2.2 alias() . . . 4
2.3 filter() . . . 4
2.4 withColumnRenamed() . . . 5
2.5 withColumn() . . . 6
2.6 Type casting . . . 7
2.7 sort() . . . 8
2.8 limit() . . . 9
2.9 drop() . . . 9
2.10 dropDuplicates() and distinct() . . . 10
From notebook to pipeline . . . 12
Glossary . . . 12
Check yourself . . . 14
Answers and solutions . . . 15

From here on the notebook transforms the BigMart DataFrame `df` that was read with the DDL schema. Remember from Phase 0: a transformation returns a **new** DataFrame, and `.display()` is the action that shows it.

THREE ILLUSTRATIVE BIGMART ROWS USED IN THE EXAMPLES BELOW

Item_Identifier	Item_Weight	Item_Fat_Content	Item_Type	Item_MRP	Outlet_Size
FDA15	9.3	Low Fat	Dairy	249.8092	Medium
DRC01	5.92	Regular	Soft Drinks	48.2692	Medium
FDN15	17.5	Low Fat	Meat	141.618	Medium

Only a few columns are shown and the rows are for illustration; your own display() shows the full dataset.

2.1 select()

`select` chooses which columns to keep, like the column list of a SQL `SELECT`. Wrapping a name in `col()` tells Spark it is a column.

BEFORE: DF

Item_Id	Item_Weight	Fat_Content	Item_Type	Item_MRP
FDA15	9.3	Low Fat	Dairy	249.8
DRC01	5.92	Regular	Soft Drinks	48.3
FDN15	17.5	Low Fat	Meat	141.6

→

AFTER (3 COLUMNS KEPT)

Item_Id	Item_Weight	Fat_Content
FDA15	9.3	Low Fat
DRC01	5.92	Regular
FDN15	17.5	Low Fat

```
df.select(col('Item_Identifier'), col('Item_Weight'), col('Item_Fat_Content'))
```
select keeps only the columns you name (greyed-out columns are left out of the result).

PYSPARK

```
df.display()
```

DECODE THE CODE

`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.

PYSPARK

```
df.select(col('Item_Identifier'),col('Item_Weight'),col('Item_Fat_Content')).display()
```

DECODE THE CODE

`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).

SAMPLE OUTPUT

Item_Identifier	Item_Weight	Item_Fat_Content
FDA15	9.3	Low Fat
DRC01	5.92	Regular
FDN15	17.5	Low Fat

THREE WAYS TO NAME A COLUMN

`col('Item_Weight')`, `df['Item_Weight']` and the plain string `'Item_Weight'` all work in select. Use `col()` when you want to apply an operation (`.alias`, `.cast`, `> 10`).

2.2 alias()

`alias` renames a column **inside the result of that select** - the original DataFrame is unchanged.

PYSPARK

```
df.select(col('Item_Identifier').alias('Item_ID')).display()
```

DECODE THE CODE

`.alias('new')`	Gives a column a new name **in this result only**.

PYSPARK

```
df.display()
```

SAMPLE OUTPUT

Item_ID
FDA15
DRC01
FDN15

The second cell, `df.display()`, proves the point: the original still shows `Item_Identifier`.

2.3 filter()

`filter` keeps only the rows where a condition is true (like SQL `WHERE`). `where()` is an exact synonym.

DF.FILTER(COL('ITEM_FAT_CONTENT') == 'REGULAR')

Item_Id	Item_Weight	Fat_Content	Item_Type	Item_MRP	
~~FDA15~~	~~9.3~~	~~Low Fat~~	~~Dairy~~	~~249.8~~	removed (False)
DRC01	5.92	Regular	Soft Drinks	48.3	kept (True)
~~FDN15~~	~~17.5~~	~~Low Fat~~	~~Meat~~	~~141.6~~	removed (False)
FDX07	19.2	Regular	Fruits	182.1	kept (True)

RESULT

Item_Id	Item_Weight	Fat_Content	Item_Type	Item_MRP
DRC01	5.92	Regular	Soft Drinks	48.3
FDX07	19.2	Regular	Fruits	182.1

Rows whose condition is false are dropped; the columns stay as they were.

Scenario 1 - one condition

PYSPARK

```
df.filter(col('Item_Fat_Content')=='Regular').display()
```

DECODE THE CODE

`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`==`	**Equals** test. A single **=** would assign instead of compare.

Note the **double equals** `==` for comparison - a single `=` would be an assignment.

Scenario 2 - two conditions with AND

PYSPARK

```
df.filter((col('Item_Type') == 'Soft Drinks') & (col('Item_Weight')<10)).display()
```

DECODE THE CODE

`& |`	**AND** / **OR** between conditions. Put each condition in its own ( ).

Scenario 3 - null check and a list of values

PYSPARK · LINE BREAK ADDED FOR THE PAGE, CODE UNCHANGED

```
df.filter((col('Outlet_Size').isNull()) &
          (col('Outlet_Location_Type').isin('Tier 1','Tier 2'))).display()
```

DECODE THE CODE

`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)
`.isin(a, b)`	True when the value is in the given list.

You want	Write	Not
AND	`&`	`and`
OR	`|`	`or`
NOT	`~`	`not`
Value is missing	`col('x').isNull()`	`col('x') == None`
Value is one of a list	`col('x').isin('a', 'b')`	a chain of ORs

ALWAYS PUT EACH CONDITION IN ITS OWN PARENTHESES

In Python `&` binds tighter than `==` and `<`. Without brackets, `col('a') == 1 & col('b') < 10` is read in the wrong order and raises an error. That is why every condition in scenarios 2 and 3 is wrapped in `( )`.

SCENARIO 3 IN WORDS

Outlets whose size is **missing** (null) **and** whose location type is Tier 1 or Tier 2. Nulls need `isNull()` because a null is never equal to anything, even to another null.

2.4 withColumnRenamed()

PYSPARK

```
df.withColumnRenamed('Item_Weight','Item_Wt').display()
```

DECODE THE CODE

`.withColumnRenamed(old, new)`	Renames a column in the DataFrame; everything else is untouched.

Arguments are `(existing_name, new_name)`. Every other column is left as it is. Like all transformations it returns a new DataFrame; the notebook only displays it, so `df` still has `Item_Weight`. Unlike `alias`, this works on the whole DataFrame, not inside a select.

2.5 withColumn()

`withColumn(name, expression)` **adds** a new column, or **replaces** an existing one if the name already exists. It is the workhorse of PySpark transformations.

DF.WITHCOLUMN('FLAG', LIT('NEW'))

Item_Id	Item_Weight	Item_MRP
FDA15	9.3	249.8
DRC01	5.92	48.3
FDN15	17.5	141.6

→

new column

Item_Id	Item_Weight	Item_MRP	flag
FDA15	9.3	249.8	new
DRC01	5.92	48.3	new
FDN15	17.5	141.6	new

withColumn(name, expression): new name -> adds a column. existing name -> replaces it.
lit('new') makes a constant; col('A') * col('B') makes a calculated column.

Scenario 1 - add a constant column, and a calculated column

PYSPARK

```
df = df.withColumn('flag',lit("new"))
```

DECODE THE CODE

`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).

PYSPARK

```
df.display()
```

PYSPARK

```
df.withColumn('multiply',col('Item_Weight')*col('Item_MRP')).display()
```

SAMPLE OUTPUT

Item_Identifier	Item_Weight	Item_MRP	flag	multiply
FDA15	9.3	249.8092	new	2323.23
DRC01	5.92	48.2692	new	285.75
FDN15	17.5	141.618	new	2478.32

Illustrative rows; the multiply column is rounded here for readability.

- `lit('new')` turns a plain Python value into a column. Without `lit`, Spark would look for a *column* called new.
- The first cell is assigned back (`df = df.withColumn(...)`) so `df` now permanently has the `flag` column.
- The second cell is **not** assigned, so `multiply` exists only in that displayed result.
- `Item_Weight` is text (STRING) in this schema; Spark converts it to a number automatically for the multiplication. Rows where it is null give a null result.

Scenario 2 - clean values with regexp_replace

PYSPARK

```
df = df.withColumn('Item_Fat_Content',regexp_replace(col('Item_Fat_Content'),"Regular","Reg"))\
    .withColumn('Item_Fat_Content',regexp_replace(col('Item_Fat_Content'),"Low Fat","Lf"))

df.display()
```

DECODE THE CODE

`regexp_replace(col, pattern, new)`	Finds text matching a pattern and swaps it for new text.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.

SAMPLE OUTPUT

Item_Fat_Content (before)	Item_Fat_Content (after)
Regular	Reg
Low Fat	Lf

`regexp_replace(column, pattern, replacement)` finds text that matches a pattern and swaps it. Here two `withColumn` calls are chained: the first changes Regular to Reg, the second changes Low Fat to Lf. Because the new column has the **same name**, it replaces the old one. The result is assigned back to `df`.

> **THIS CHANGE MATTERS LATER**
> Item_Fat_Content now contains **Reg** and **Lf**. Phase 7's SQL query filters on `'Lf'` - it only works because of this cell. Running cells out of order is the commonest source of confusing results in a notebook.

2.6 Type casting

Casting converts a column from one data type to another. Use `withColumn` with the column's own `.cast(...)`.

PYSPARK

```
df = df.withColumn('Item_Weight', col('Item_Weight').cast(StringType()))
```

DECODE THE CODE

`.cast(type)`	Converts a column to another data type.
`StringType() / IntegerType() / DoubleType()`	Type objects for text, whole numbers and decimal numbers.

PYSPARK

```
df.printSchema()
```

DECODE THE CODE

`printSchema()`	Prints each column's name, type and whether it allows nulls - as a tree. Metadata only, so it is instant.

Cast to	Write
Text	`.cast(StringType())` or `.cast('string')`
Whole number	`.cast(IntegerType())` or `.cast('int')`
Decimal number	`.cast(DoubleType())` or `.cast('double')`

NOTEBOOK OBSERVATION

The notebook casts Item_Weight to **StringType**, but the DDL schema already declared it as STRING, so this particular cast changes nothing - it is a demonstration of the syntax. To make the weight genuinely numeric you would cast to a number instead (below).

A CAST THAT WOULD MAKE SORTING AND MATHS RELIABLE

PYSPARK · EXTRA

```
df = df.withColumn('Item_Weight', col('Item_Weight').cast(DoubleType()))
```

If a value cannot be converted (for example the text "abc" cast to a number) the result depends on the Spark version: older Spark 3 setups return **null** for that row, but **Spark 4 (ANSI mode on by default) raises an error** (CAST_INVALID_INPUT). `try_cast` (section 3.7) returns null on both.

2.7 sort()

`sort` orders the rows (like SQL `ORDER BY`); `orderBy` is a synonym. Ascending is the default.

INPUT (TEXT)

Item_Weight
9.3
5.92
17.5
19.2

sort desc →

AS TEXT (CHARACTERS)

Item_Weight
9.3
5.92
19.2
17.5

`"9" > "5" > "1"` *compares one character at a time*

AS NUMBERS (AFTER CAST)

Item_Weight
19.2
17.5
9.3
5.92

numeric order

```
sort(['Item_Weight','Item_Visibility'], ascending=[0,1])
```

1st column is the main order; the 2nd only breaks ties. 0 = descending, 1 = ascending.
Ascending puts nulls first; descending puts nulls last.

Scenario 1 - descending

PYSPARK

```
df.sort(col('Item_Weight').desc()).display()
```

DECODE THE CODE

`.sort(...)`	Orders the rows - like SQL ORDER BY. Ascending by default.
`.desc() / .asc()`	Sort direction for one column: descending / ascending.

Scenario 2 - ascending

PYSPARK

```
df.sort(col('Item_Visibility').asc()).display()
```

Scenario 3 - two columns, both descending

PYSPARK

```
df.sort(['Item_Weight','Item_Visibility'],ascending = [0,0]).display()
```

DECODE THE CODE

`ascending=[0,1]`	One flag per sort column, in order: **0 = descending**, **1 = ascending**.

Scenario 4 - two columns, mixed directions

PYSPARK

```
df.sort(['Item_weight','Item_Visibility'], ascending = [0,1]).display()
```

- With a list of columns, the **first** column is the main sort; the second only breaks ties among equal values of the first.
- `ascending=[0,1]` lines up with the column list: **0 = descending** (False), **1 = ascending** (True). So scenario 4 means weight descending, then visibility ascending.
- Scenario 4 spells `'Item_weight'` with a lowercase w. It still works because Spark matches column names **without regard to case** by default.

ITEM_WEIGHT IS TEXT, SO IT SORTS ALPHABETICALLY

Under the DDL schema Item_Weight is a STRING. A descending sort compares characters, so **"9.3"** comes before **"17.5"** because "9" is greater than "1". Cast to double first (see 2.6) when you want numeric order. `Item_Visibility` is a DOUBLE, so scenario 2 is numeric.

WHERE DO NULLS GO?

Ascending puts nulls **first**; descending puts them **last**. So the null weights sit at the bottom of scenario 1 and the top of an ascending sort.

2.8 limit()

PYSPARK

```
df.limit(10).display()
```

DECODE THE CODE

`.limit(n)`	Keeps at most the first n rows.

`limit(n)` keeps at most the first n rows. It is a quick way to inspect a huge DataFrame. Combine it with `sort` for a top-n: `df.sort(col('Item_MRP').desc()).limit(10)` gives the ten highest-priced rows. Without a sort, *which* n rows you get is not guaranteed.

2.9 drop()

Scenario 1 - drop one column

PYSPARK

```
df.drop('Item_Visibility').display()
```

DECODE THE CODE

`.drop('a','b')`	Removes the named column(s) from the result.

Scenario 2 - drop several columns

PYSPARK

```
df.drop('Item_Visibility','Item_Type').display()
```

`drop` removes the named columns (the opposite of `select`). Pass as many names as you like. Naming a column that does not exist is not an error. Neither cell is assigned, so `df` keeps all its columns.

DF

Item_Id	Item_Visibility	Item_MRP
FDA15	0.016	249.8
DRC01	0.019	48.3

→ `.drop('Item_Visibility')` →

NEW DATAFRAME (A COPY)

Item_Id	Item_MRP
FDA15	249.8
DRC01	48.3

df is NOT changed - it still has Item_Visibility.

To keep the change you must assign it back: `df = df.drop('Item_Visibility')`

2.10 dropDuplicates() and distinct()

ORIGINAL ROWS

Item_Id	Item_Type	Fat_Content
FDA15	Dairy	Low Fat
FDA15	Dairy	Low Fat
DRC01	Soft Drinks	Regular
FDN15	Dairy	Low Fat
DRC01	Meat	Regular

DROPDUPLICATES() - WHOLE ROW MUST MATCH

Item_Id	Item_Type	Fat_Content
FDA15	Dairy	Low Fat
DRC01	Soft Drinks	Regular
FDN15	Dairy	Low Fat
DRC01	Meat	Regular

row 2 was an exact copy of row 1

DROP_DUPLICATES(SUBSET=['ITEM_TYPE'])

Item_Id	Item_Type	Fat_Content
FDA15	Dairy	Low Fat
DRC01	Soft Drinks	Regular
DRC01	Meat	Regular

one row per different Item_Type

distinct() = dropDuplicates() with no subset. Which duplicate survives a subset is not guaranteed.

Scenario 1 - remove fully identical rows

PYSPARK

```
df.dropDuplicates().display()
```

DECODE THE CODE

`.dropDuplicates()`	Removes repeated rows (same as *drop_duplicates*). By default whole rows must match.

Scenario 2 - one row per value of chosen columns

PYSPARK

```
df.drop_duplicates(subset=['Item_Type']).display()
```

DECODE THE CODE

`subset=[...]`	Restricts the operation to the listed columns only.

PYSPARK

```
df.distinct().display()
```

DECODE THE CODE

`.distinct()`	Unique rows across all columns.

Call	Result
`df.dropDuplicates()`	Removes rows where **every** column is identical to another row.
`df.dropDuplicates(subset=['Item_Type'])`	Keeps **one row for each different Item_Type**. Which row survives is not guaranteed.
`df.drop_duplicates(...)`	Exactly the same as `dropDuplicates` (Python-style alias).
`df.distinct()`	Same as `dropDuplicates()` with no subset: unique full rows.

TO LIST THE UNIQUE VALUES OF ONE COLUMN

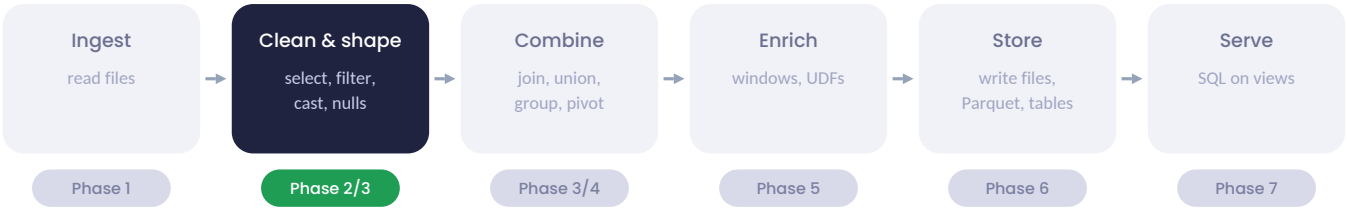
Use `df.select('Item_Type').distinct()`. `distinct()` cannot take a subset, but selecting the column first achieves the same.

PHASE 2 CHECKPOINT

- I can pick, rename and drop columns with select / alias / withColumnRenamed / drop.
- I can filter with &, |, isNull() and isin(), with each condition in parentheses.
- I can add and replace columns with withColumn, lit and regexp_replace, and I remember to assign the result back.
- I can cast types and I know Item_Weight is text in this notebook.
- I can sort on one or several columns in mixed directions and use limit for a top-n.
- I can remove duplicates for whole rows or for a subset of columns.

From notebook to pipeline

Data engineers use these same skills inside **pipelines**. Here is where this phase sits, and how it is used in practice.



DATA ENGINEERING LENS

This is the **cleaning layer**: standardise values (regexp_replace turns Regular into Reg), fix types (cast), remove duplicates, and drop columns you do not need.

Every transformation returns a new DataFrame, so a pipeline is a **chain of small steps**. You can run .display() after any step to check it.

Selecting and filtering early keeps the data small before the expensive steps (groupBy, join, sort) that come later.

Remember to assign results back (df = df.withColumn(...)); an unassigned step changes nothing.

Glossary

Concepts from this phase, then every **code term** the decoder explained, in the order you met them.

Concept	In plain words
Immutable	A DataFrame never changes; every transformation returns a new one.
Expression	A calculation on columns, such as col('a') * col('b').
Condition	A True/False test on each row, used by filter.
Casting	Converting a column from one data type to another.
Duplicate row	A row identical to another in all columns (or in the chosen subset).

Code terms decoded in this phase

Code term	Meaning
`.display()`	Databricks' table viewer. It is an **action**, so it makes Spark run the job.
`.select(...)`	Keeps only the listed columns (or new calculated columns) - like SQL SELECT.
`col('name')`	Refers to a column by name so you can do something with it (rename, compare, calculate).
`.alias('new')`	Gives a column a new name **in this result only**.
`.filter(condition)`	Keeps the rows where the condition is true - like SQL WHERE.
`==`	**Equals** test. A single **=** would assign instead of compare.
`& |`	**AND** / **OR** between conditions. Put each condition in its own ( ).
`.isNull()`	True where the value is missing. (A null is never *equal* to anything, so == *None* fails.)
`.isin(a, b)`	True when the value is in the given list.
`.withColumnRenamed(old, new)`	Renames a column in the DataFrame; everything else is untouched.
`.withColumn(name, expression)`	**Adds** a column, or **replaces** the one with the same name.
`lit(value)`	Turns a plain Python value into a constant column (otherwise Spark looks for a column with that name).
`regexp_replace(col, pattern, new)`	Finds text matching a pattern and swaps it for new text.
`\ (backslash)`	Line continuation: the statement carries on in the next line. Nothing may follow it on the same line.
`.cast(type)`	Converts a column to another data type.
`StringType() / IntegerType() / DoubleType()`	Type objects for text, whole numbers and decimal numbers.
`printSchema()`	Prints each column's name, type and whether it allows nulls - as a tree. Metadata only, so it is instant.
`.sort(...)`	Orders the rows - like SQL ORDER BY. Ascending by default.
`.desc() / .asc()`	Sort direction for one column: descending / ascending.
`ascending=[0,1]`	One flag per sort column, in order: **0 = descending**, **1 = ascending**.
`.limit(n)`	Keeps at most the first n rows.
`.drop('a','b')`	Removes the named column(s) from the result.
`.dropDuplicates()`	Removes repeated rows (same as *drop_duplicates*). By default whole rows must match.
`subset=[...]`	Restricts the operation to the listed columns only.
`.distinct()`	Unique rows across all columns.

Check yourself

Try each question first, then click **Show answer**. Every answer has a **back** link that returns you to the question.

Questions

Q1 Why must each condition in `(col('Item_Type') == 'Soft Drinks') & (col('Item_Weight') < 10)` be in parentheses? Show answer

Q2 What does `withColumn` do when the column name already exists? Show answer

Q3 What does `df.sort(['Item_Weight','Item_Visibility'], ascending=[0,1])` do? Show answer

Q4 In the notebook a descending sort puts "9.3" before "17.5". Why? Show answer

Q5 How does `dropDuplicates(subset=['Item_Type'])` differ from `distinct()`? Show answer

Hands-on tasks

Task 1 Show only Item_Identifier and Item_MRP of the 'Regular' items (before the regexp_replace step), with Item_MRP renamed to MRP. Show solution

Task 2 Cast Item_Weight to a number and show the five heaviest items. Show solution

Answers and solutions

A1 Python's & binds tighter than == and <, so without brackets the expression is evaluated in the wrong order and raises an error. back to question

A2 It **replaces** that column. If the name is new it adds a column. back to question

A3 Sorts by Item_Weight **descending** (0) first; rows with equal weight are then ordered by Item_Visibility **ascending** (1). back to question

A4 Item_Weight is a STRING under the DDL schema, so it is compared character by character and "9" is greater than "1". Cast to double for numeric order. back to question

A5 distinct removes rows that are identical in **all** columns. The subset version keeps one row per different Item_Type (which row survives is not guaranteed). back to question

Task solutions

Solution 1 back to task

SOLUTION

```
df.filter(col('Item_Fat_Content') == 'Regular')\
  .select('Item_Identifier', 'Item_MRP')\
  .withColumnRenamed('Item_MRP', 'MRP')\
  .display()
```

Solution 2 back to task

SOLUTION

```
df.withColumn('Item_Weight', col('Item_Weight').cast(DoubleType()))\
  .sort(col('Item_Weight').desc())\
  .limit(5)\
  .display()
```

PREVIOUS
← Phase 1 · Reading Data

NEXT
Phase 3 · Intermediate Transformations →

Tip: keep all nine PDFs in the same folder so the previous / next links open the right file.